API 계약과 LLM 서비스 구현

WEEK 09 · 2026.10.30 · 온라인 실시간

입출력 계약 · 실패 복구 · 비교 실험

차성재 · 무신사 Core AI PM × AI 대학원 겸임교수

이번 주의 완료 증거

01	**계약**	정상 응답·입력 오류를 구분하는 API
02	**복구**	시간 초과·거절·중단 이후 자원 해제
03	**실험**	조건·표본·성공 지연·오류율의 기록

API · Application Programming Interface · 응용 프로그래밍 인터페이스

요청·응답 계약 · 비동기 처리 · 실패 복구 · 비교 실험

16주 로드맵 · 9주차

01–03	09.04–09.18	문제·프롬프트·평가
04–05	09.25–10.02	검색·근거
06	10.09	휴일·수업 없음
07–08	10.16–10.23	기획 리뷰·제출 평가
09	10.30	**API 계약·실패 복구**
10	11.06	배포·실행 관찰
11–12	11.13–11.20	오프라인·온라인 평가
13–14	11.27–12.04	안전·비용·용량
15	12.11	통합 점검·리허설
16	12.18	실시간 온라인 발표

기획서에서 실행 계약까지

01

요구사항

사용자·실패

02

API 명세

입력·출력

03

검증 코드

성공·오류

04

실행 근거

판단·한계

8주차 문서를 다시 쓰기보다, 요구사항 하나를 실행 가능한 기능으로 연결

요청·응답 계약 · 비동기 처리 · 실패 복구 · 비교 실험

100분 수업 구성

시간	학습·활동
00–20분	문제·API 계약·실패 응답
20–40분	비동기·용량·캐시·추론 구조
40–45분	휴식·환경·커널 확인
45–85분	선택 핵심 셀·브라우저·실패 복구
85–100분	부하 결과·프로젝트 리뷰

전체 Notebook 85분 분량 · 수업은 핵심 셀, 나머지는 복습

실행 중인 반품 안내 서비스

한 명의 질문에는 정상 응답

동시 접속에서는 대기·중단·잘못된 입력이 발생

오늘의 대상은 답변 함수 주변의 서비스 동작

합성 정책·고정 응답으로 운영 구조를 실험 · 실제 고객 정책 아님

노트북 함수와 서비스 API

노트북 함수

개발자가 입력·예외를 직접 확인

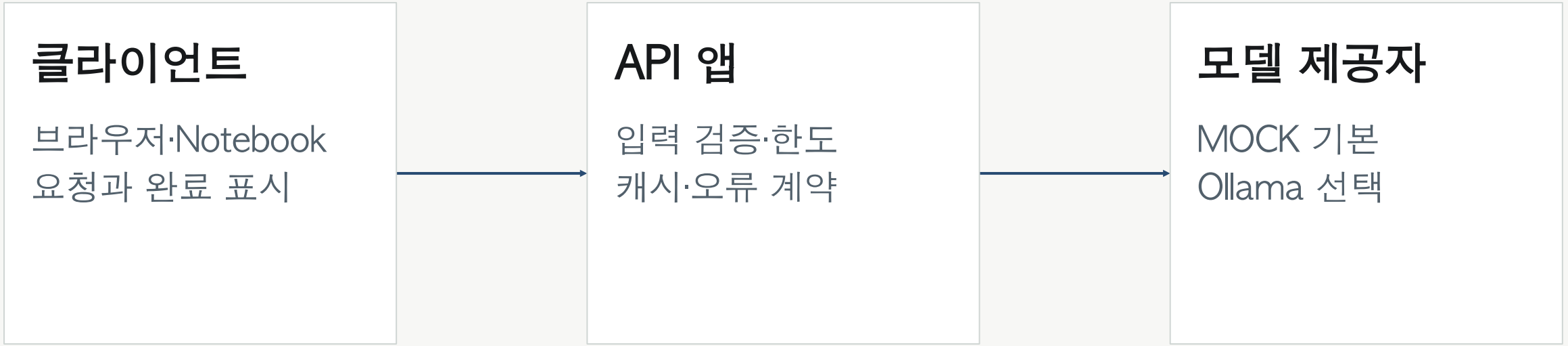
커널 안의 한 실행 흐름

서비스 API

서로 다른 클라이언트의 동시 요청

명시적 계약·실패 응답·자원 한도

클라이언트·API·모델의 경계



Uvicorn: HTTP 요청을 앱에 전달하는 ASGI 서버

이번 실습은 한 프로세스 · 외부 모델은 선택 · 인증·영구 저장은 미구현

요청·응답 계약 · 비동기 처리 · 실패 복구 · 비교 실험

한 요청의 왕복 경로



검증 실패는 모델 호출 전 종료 · 요청 ID로 결과와 메타데이터 연결

오늘의 실습 화면



요청 조건

메시지·캐시·장애 주입

응답 계약

상태·요청 ID·버전

실행 경계

MOCK 고정 응답 표시

실제 로컬 실행 화면 · 세부 JSON은 다음 장표에서 확대

요청·응답 계약

정상 입력만 정의하는 명세의 한계

입력 검증·상태 코드·버전·완료 조건

HTTP 요청의 구성

```
POST /api/v1/chat
Content-Type: application/json

{"message": "반품 정책",
 "max_tokens": 64,
 "use_cache": true}
```

전송 계약의 분리

경로·메서드·헤더·본문

JSON 문법과 업무 의미는 별도

HTTP · Hypertext Transfer Protocol · JSON · JavaScript Object Notation

Pydantic 입력 모델

```
class ChatRequest(BaseModel):
    model_config = ConfigDict(extra="forbid", strict=True)
    message: str = Field(min_length=1, max_length=1000)
    max_tokens: int = Field(default=64, ge=8, le=128)
```

공백 검사 추가 · 숫자 문자열·임의 필드 거부 · app.py의 실제 계약과 연결

필드의 의미와 한도

필드	계약	확인
message	1–1000자·공백 제거	공백만 있으면 422
max_tokens	정수 8–128	MOCK 토큰 수 보장 아님
use_cache	불리언	일반 응답만 적용
scenario	MOCK 장애 주입	실제 제공자는 normal만

문자 수·토큰 수·바이트 크기는 서로 다름

422 오류의 안전한 응답

```
{"error": {
  "code": "validation_error",
  "fields": [{
    "location": ["body", "message"],
    "type": "string_too_short"
  }]
}, "request_id": "..."}
```

수정 가능한 입력 오류

원문 입력은 오류에 복사하지 않음

클라이언트는 입력 수정 안내

422 · 입력 검증 실패 · 무조건 재시도로 해결하지 않음

정상 응답의 버전 정보

```
{"request_id": "...",
 "output": "[MOCK] ...",
 "provider": "mock",
 "model": "teaching-mock-v1",
 "prompt_version": "v1",
 "cached": false,
 "simulated": true}
```

다시 확인 가능한 결과

상태 200과 의미 정확성은 별개

버전·캐시·모사 여부를 함께 표시

response_model=ChatResponse · 응답에 허용할 필드 명시

프로세스 상태와 준비 상태

GET /health

프로세스가 응답하는지 확인

모델의 존재·생성 성공 보장 없음

GET /ready

MOCK 준비 또는 Ollama 설치 확인

실제 생성 성공은 별도 점검

OpenAPI와 클라이언트 검사

01

명세 생성

/openapi.json

02

수동 확인

/docs

03

반복 검사

HTTPX

04

변경 검토

호환성

OpenAPI · API 명세 형식 · 문서 화면과 자동 테스트를 함께 활용

요청·응답 계약 · 비동기 처리 · 실패 복구 · 비교 실험

경로 버전과 동작 버전

/api/v1

클라이언트 입출력 계약의 버전

삭제·타입 변경은 호환성 검토

prompt_version · model

같은 API 내부의 생성 동작

캐시·평가·비교 조건에 반영

서버 없이 가능한 계약 검사

```
app = create_app()
transport = httpx.ASGITransport(app=app)
async with httpx.AsyncClient(
    transport=transport, base_url="http://lab") as client:
    r = await client.post("/api/v1/chat",
                          json={"message": "반품 정책"})
assert r.status_code == 200
```

Notebook 01 · 실제 네트워크·Uvicorn 성능 측정과 구분

오류 코드와 다음 행동

상태	실습 의미	다음 행동
422	입력 계약 위반	필드·길이·타입 수정
429	요청 빈도 제한	빈도·재시도 예산 조정
502	상류 응답 오류	모델명·상류 응답 확인
503	용량 초과·연결 불가	용량·포트·제공자 확인
504	제한시간 초과	시간 예산·상류 생존 확인

같은 상태의 원인은 error.code로 구분 · Retry-After는 보장 시간이 아님

요청 ID와 원문 없는 추적

01

요청 수신

ID 발급

02

처리 결과

상태·시간

03

최근 기록

최대 200건

04

원인 확인

코드·버전

메시지·답변·연락처는 기록하지 않음 · 메모리 캐시의 답변 보유는 별도

동시 요청과 실패 복구

기다리는 작업과 계산하는 작업의 차이

요청 유입·처리 용량·시간 예산의 분리

순차 대기와 동시 대기



30ms 대기 4건의 개념도 · 실제 GPU 계산 속도와 무관

비동기 I/O 실행

```
async def io_job():
    await asyncio.sleep(0.03)
    return "done"

results = await asyncio.gather(
    *(io_job() for _ in range(4)))
```

I/O · Input/Output · 입출력 / ASGI · Asynchronous Server Gateway Interface

async def와 블로킹 호출

협력 가능한 대기

await client.post(...)

I/O 대기 중 다른 요청 진행

이벤트 루프를 막는 호출

time.sleep(...) · 동기 HTTP 호출

async def 안이라고 자동 비동기 아님

작업 성격과 실행 위치

작업	예시	선택 검토
I/O 대기	외부 HTTP·저장소	비동기 클라이언트
동기 I/O	기존 동기 라이브러리	제한된 스레드
CPU 계산	Python 중심 반복 계산	프로세스·별도 작업자
GPU 추론	언어 모델 실행	모델 서버·스케줄러

CPU · Central Processing Unit / GPU · Graphics Processing Unit

CPython 3.11과 GIL

GIL · Global Interpreter Lock · 전역 인터프리터 잠금

일반 Python 바이트코드의 스레드 병렬 실행에 제약

I/O·네이티브 연산·별도 프로세스는 조건을 나누어 검토

이 강좌 환경은 CPython 3.11 · 다른 런타임·버전의 조건을 혼합하지 않음

처리 용량과 즉시 거절



max_inflight=2 · 무제한 대기열 없음 · 빈도 초과는 별도 429

빈도 제한과 동시 처리 제한

빈도 제한 · 429

일정 시간에 들어오는 요청 제어

토큰 버킷의 보충률·burst

동시 처리 제한 · 503

실행 중인 작업 수 제어

max_inflight · 종료 후 해제

클라이언트와 서버의 시간 예산

전체 요청 예산 ≥ 접속·대기·생성·전송

서버 제한시간과 클라이언트 대기시간을 각각 명시

외부 작업 취소는 별도의 확인 대상

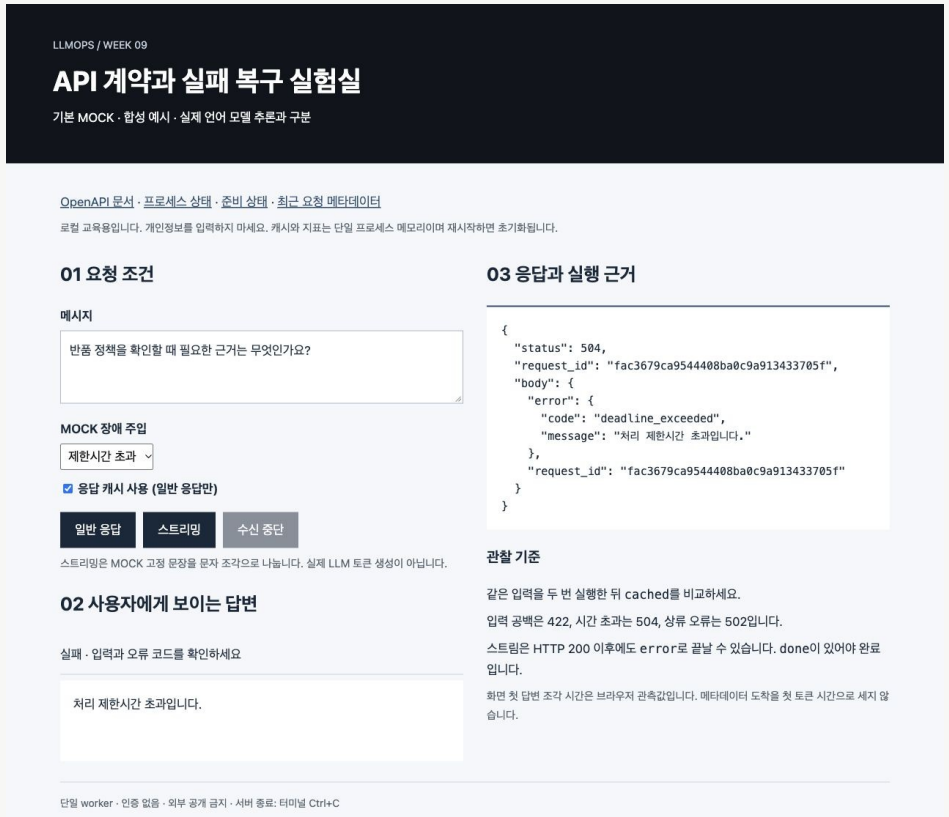
MOCK 기본 제한 0.3초 · 실제 Ollama 경로는 30초 · 서비스 목표값 아님

제한시간과 자원 해제

```
try:
    async with asyncio.timeout(self.settings.timeout_s):
        output = await self.generate(req)
except TimeoutError:
    # 504 오류 계약
    ...
finally:
    self.active -= 1
```

서비스 메서드의 개념 발췌 · 정상·오류·취소 모두 해제 · 전체 함수는 service.py

시간 초과의 실제 화면



선택 조건

MOCK 제한시간 초과

확인 결과

504 · deadline_exceeded

복구 확인

active=0 후 다음 요청

실행 캡처 · 정상값으로 오류를 숨기지 않는 계약

재시도 정책의 검토

상황	검토	금지할 동작
422	입력 수정	같은 입력 무한 반복
429·503	안내·백오프·횟수 상한	즉시 반복으로 부하 증폭
504	중복 실행·과금·취소	원래 작업 종료로 단정
스트림 중단	부분 답변·완료 상태	이어 붙여 완성으로 표시

이번 앱은 자동 재시도 미구현 · 재시도는 비용·중복·시간 예산과 함께 결정

취소 이후의 용량 검사

```
task = asyncio.create_task(service.chat(req, "cancel-demo"))
await asyncio.sleep(0.001)
task.cancel()
try:
    await task
except asyncio.CancelledError:
    pass
assert service.active == 0
```

Notebook 02 · 로컬 자원 해제와 외부 GPU 작업 종료를 구분

캐시와 추론 구조

응답 재사용과 모델 내부 상태 재사용의 차이

빠른 응답의 원인과 재사용 가능한 범위

세 종류의 캐시

구분	재사용 대상	무효화 조건
응답 캐시	완성된 답변	입력·버전·권한 변경
검색 캐시	검색 결과·문서 조각	자료·권한·검색 설정 변경
KV 캐시	Attention의 Key·Value	모델·토큰·위치 변경

KV · Key–Value · 세 캐시는 별도 계층 · 이번 앱은 응답 캐시만 구현

같은 질문의 두 번째 요청



키 일치와 만료 확인 후 재사용 · 캐시 적중은 새 모델 추론이 아님

재사용 경계가 포함된 키

```
payload = {"message": req.message,
           "max_tokens": req.max_tokens,
           "provider": cfg.provider, "model": cfg.model,
           "prompt_version": cfg.prompt_version,
           "schema": "v1"}
key = sha256(canonical_json(payload)).hexdigest()
```

개념 발췌 · 실제 함수 cache_key · 인증·문서 권한이 있는 서비스는 그 경계도 포함

권한과 버전 누락의 반례

직원용 문서를 참고한 답변이 같은 질문의 캐시에 저장

일반 사용자의 질문에 그 캐시가 재사용되는 위험

인증된 권한·문서 버전 경계부터 설계

설계 반례 · 현재 실습에는 인증·다중 tenant·문서 검색이 없음

만료와 메모리 상한

TTL · Time To Live

시간 경과 후 재사용 중단

기본 30초 · 단조 시계

LRU · Least Recently Used

최근 사용하지 않은 항목부터 제거

최대 128항목 · 재시작 시 초기화

가짜 시계로 확인하는 만료

```
clock = [0.0]
cache = TTLCache(2, 2, clock=lambda: clock[0])
cache.set("A", "version1")
clock[0] = 2.0
assert cache.get("A") is None
```

Notebook 03 · 오래 기다리지 않고 만료 경계·용량 제거를 재현

콜드 캐시의 중복 요청



동시에 시작하면 두 호출 모두 실행 가능 · 진행 작업 병합은 별도 기능

입력 처리와 순차 생성

Prefill · 입력 처리

입력 토큰의 상태 계산

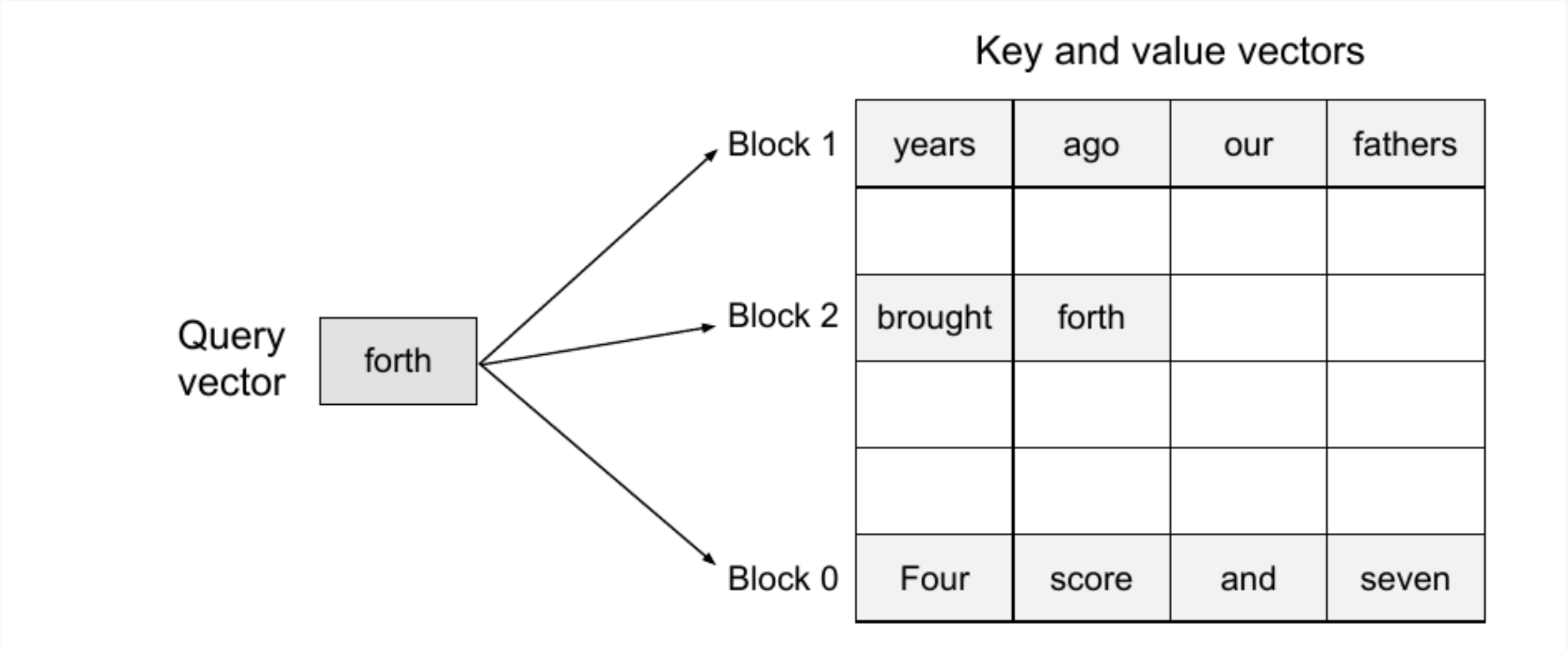
길이·배치·모델·장치에 영향

Decode · 순차 생성

이전 토큰을 참고해 다음 토큰 생성

응답 길이·메모리·스케줄링에 영향

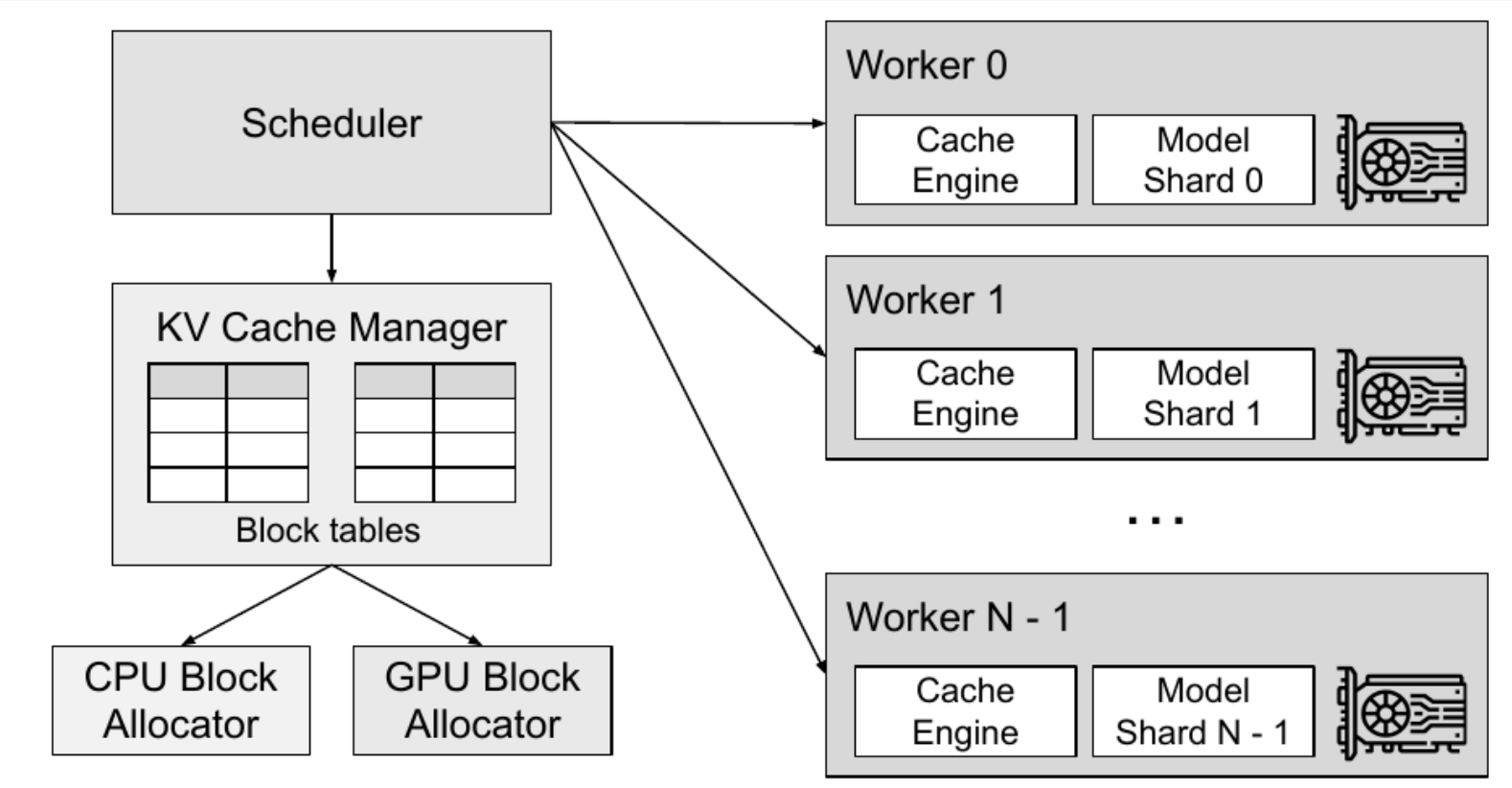
PagedAttention의 블록 참조



토큰별 Key·Value 비연속 블록의 참조 GPU 성능 검증 제외

Kwon et al., SOSP 2023, Figure 5 · CC BY 4.0 · 흑백 발췌

vLLM의 작업자와 캐시 관리



요청 실행의 조정 블록 할당과 해제 추론 엔진 내부 구조

Kwon et al., Figure 4 · CC BY 4.0 · 흑백 발췌 · 배포 성능 보장 아님

동시 API 요청과 추론 배칭

API 동시성

여러 요청이 I/O 대기를 공유

MOCK의 asyncio 실험 대상

모델 배칭

여러 시퀀스를 추론 엔진에서 스케줄

메모리·토큰·장치에 따른 별도 검증

추론 개선의 조건

선택	기대 효과	같이 볼 조건
출력 길이 제한	생성 작업 감소	답변 완전성·중단 이유
양자화	가중치 저장량 감소	품질·커널·장치
응답 캐시	반복 호출 감소	최신성·권한·유효성
추론 배칭	장치 활용·처리량	대기·메모리·개별 지연

특정 기법의 보편적 속도 우위를 가정하지 않음 · 실제 제공자 환경에서 비교

스트리밍과 완료 상태

첫 바이트·첫 내용·완료 시점의 분리

부분 답변과 실패·중단의 명시적 처리

SSE 이벤트의 진행



SSE · Server-Sent Events · 서버가 보내는 단방향 이벤트 · POST는 fetch 사용

빈 줄로 구분하는 이벤트

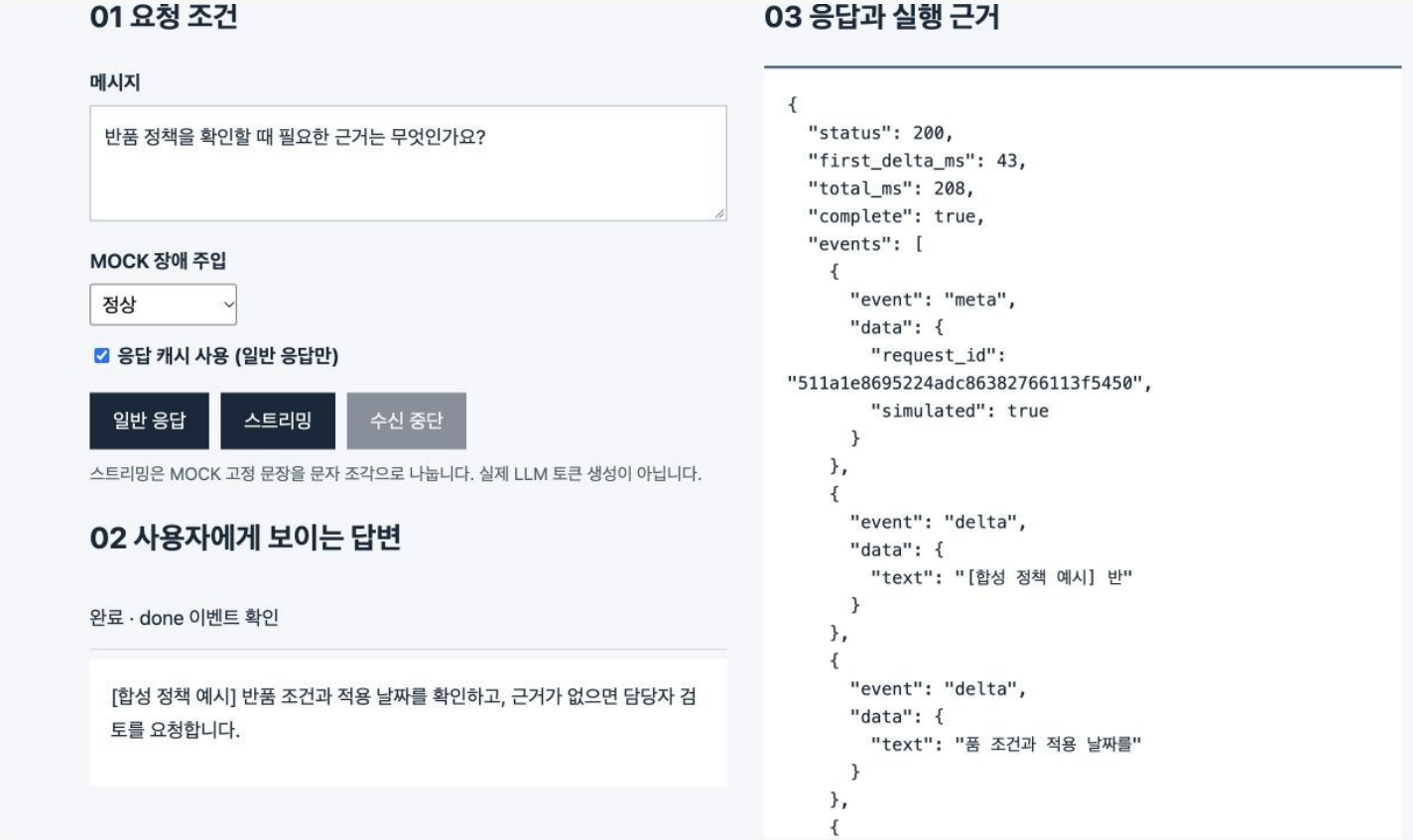
```
event: meta
data: {"request_id":"...","simulated":true}

event: delta
data: {"text":"첫 내용 조각"}

event: done
data: {"complete":true}
```

UTF-8 · text/event-stream · 네트워크 청크와 이벤트 프레임 경계는 다를 수 있음

브라우저의 스트림 수신



메타와 내용

첫 delta에서 내용 시간 측정

완료 조건

done 수신 여부

중단 동작

수신 중단·서버 용량 해제

실제 로컬 캡처 · 내용은 MOCK 문자 조각 · 토큰 생성 속도 아님

HTTP 200과 스트림 성공

응답 시작 전

입력·용량 검증 가능

422·503 등 상태 코드 결정

응답 시작 후

상태를 504로 바꿀 수 없음

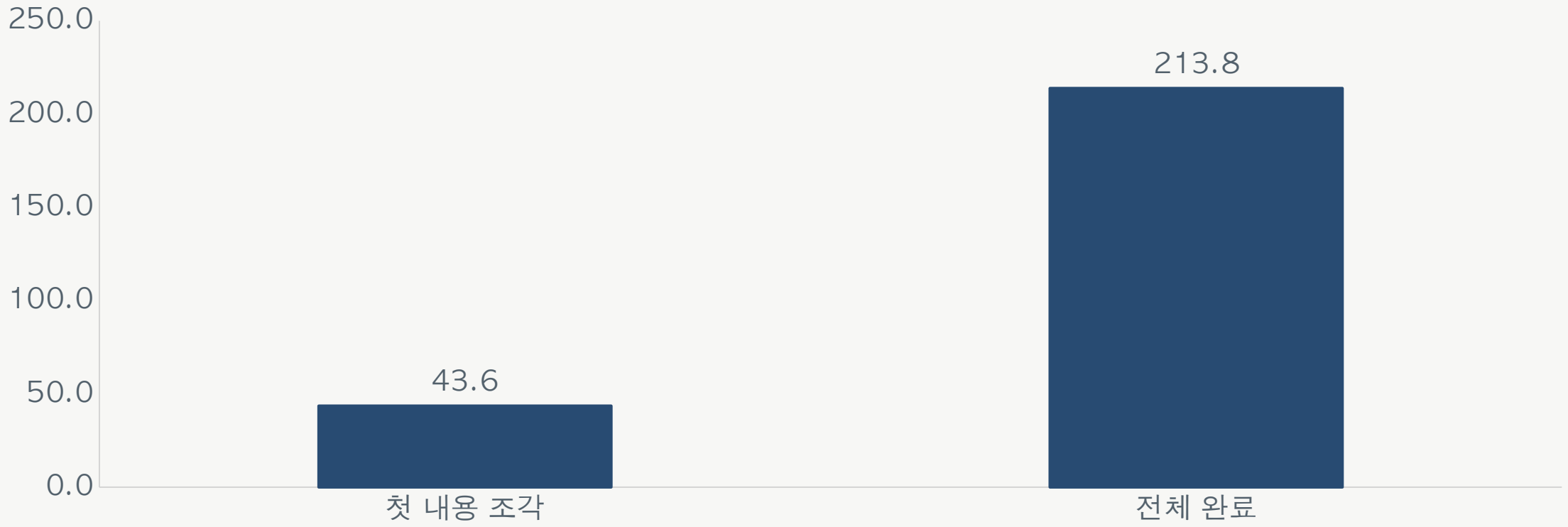
error 이벤트·미완료 상태로 전달

사용자가 보는 완료 상태

관측	판정	화면·저장
done 수신	완료	완료 표시·결과 저장
error 수신	실패·미완료	이유·다음 행동
done 없는 종료	중단·불확실	부분 답변 표시
수신 중단	사용자 취소	자원 해제 별도 확인

토큰 하나 수신했다고 전체 요청 성공으로 집계하지 않음

첫 내용 조각과 전체 완료



로컬 TCP 실측 · ms · MOCK 문자 조각 · 실제 LLM TTFT와 구분

TTFT와 스트림 측정의 경계

TTFT · Time To First Token · 첫 토큰까지의 시간

이번 MOCK은 첫 delta와 전체 완료 시간을 기록

ASGITransport는 버퍼링 가능하므로 TCP 실험에서 확인

Notebook 04는 형식 검사 · scripts/verify_week09_live.py는 실제 로컬 TCP 검사

단계별 실행과 부하 비교

예상·실행·변경·해석의 순서

성공 지연·오류율·성공 처리량의 동시 확인

다섯 Notebook의 역할

파일	실험	권장
01 API 계약	명세·정상·422·추적	15분
02 비동기·복구	503·504·502·429·취소	20분
03 캐시·버전	적중·키·TTL·중복 호출	15분
04 스트리밍	프레임·done·error·중단	15분
05 부하·프로젝트	성공 지연·오류율·요구사항	20분

각 파일 독립 실행 · 총 111셀·코드 53셀 · 수업은 핵심 셀 선택

요청·응답 계약 · 비동기 처리 · 실패 복구 · 비교 실험

VS Code 환경과 커널

```
# 저장소 루트에서 기존 환경 활성화
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pytest -q week09/lab/tests
code .
```

처음 설치: python3.11 -m venv .venv · Windows 활성화는 실습 README 참조

VS Code의 실행 순서

01	**저장소 루트**	llmops-16week 폴더 전체 열기
02	**커널 선택**	Python Environments · .venv / Python 3.11
03	**순차 실행**	위에서 아래로 실행 후 변수 하나 변경

Notebook 자동 실행은 앱 소스나 requirements를 덮어쓰지 않음

브라우저 실습 서버

```
python -m uvicorn week09.lab.app:app \
  --host 127.0.0.1 --port 8009

# 브라우저
http://127.0.0.1:8009/
http://127.0.0.1:8009/docs
```

한 worker · 외부 공개 금지 · 포트 충돌 시 8019로 변경 · 종료 Ctrl+C

40분 핵심 실습의 선택

01

계약

01 정상·422

02

복구

02 한도·초과

03

재사용

03 적중·중복

04

완료

04 error·중단

남은 시간에 05 결과 해석 · 전체 Notebook은 복습에서 확장

부하 실험의 실행 조건

```
report = await run_load(
    client, n=24, concurrency=2,
    use_cache=False, warmup=2)
display(report["summary"])
```

Notebook 05 · 동시 요청 1·2·6 비교 · 처리 용량 2 · 수면 30ms · 캐시 OFF

동시 요청 기반 부하

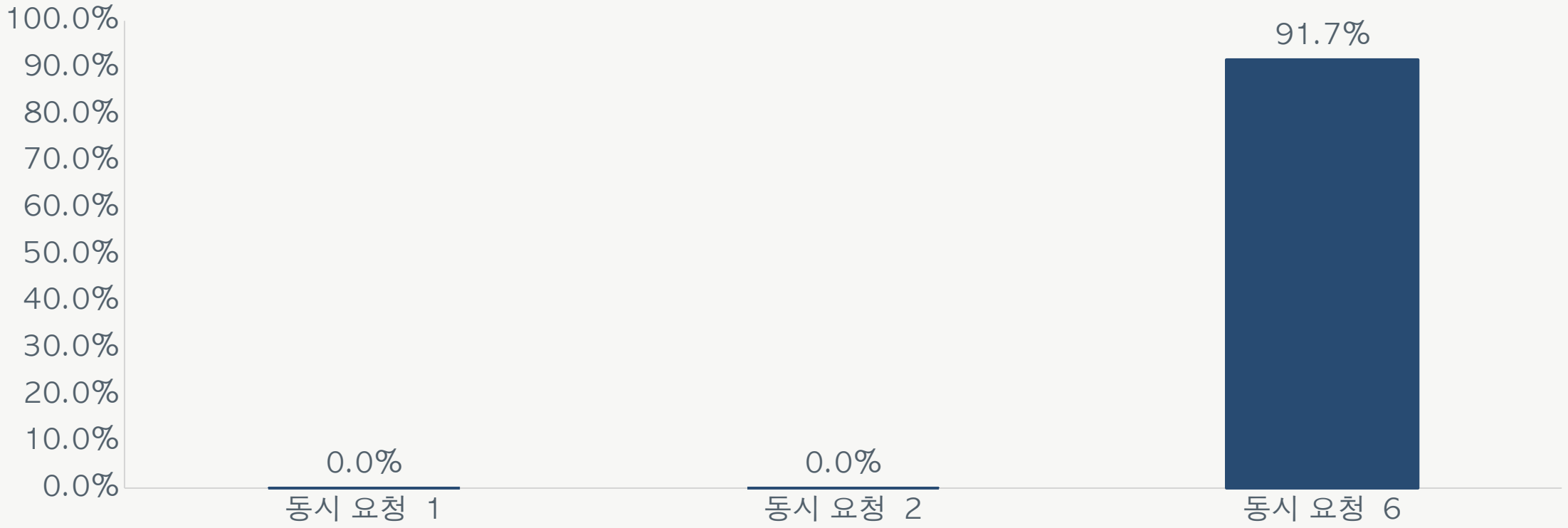
Closed-loop · 응답 후 다음 요청을 진행하는 부하

응답이 느리면 클라이언트의 다음 요청 유입도 줄어듦

고정 도착률·운영 트래픽·장시간 안정성은 별도 시험

이번 실험은 최대 100요청·동시성 8 · 외부 서비스 부하테스트 금지

동시 요청과 오류율



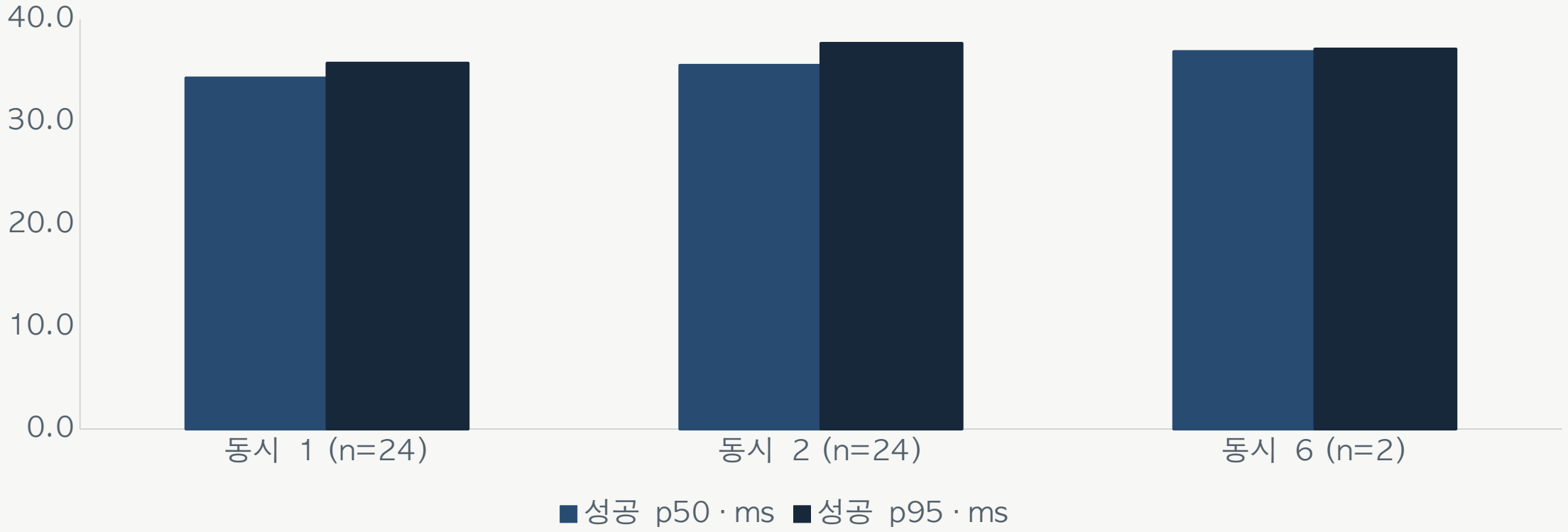
실행별 24요청 · 처리 용량 2 · 성공과 거절을 합쳐 분모 계산 · ASGI MOCK

빠른 오류가 만드는 평균 착시

가상 요청	건수	시간
성공	1	100ms
503 거절	9	각 1ms
전체 평균	10건	10.9ms
성공 p50 / 오류율	성공만 / 전체	100ms / 90%

계산 설명용 합성 값 · 거절이 빠르다고 서비스가 좋아진 것은 아님

성공 요청의 지연 분포



n은 성공 표본 수 · p50·p95 · ms · 거절은 오류율로 별도 확인

분위수와 표본 수

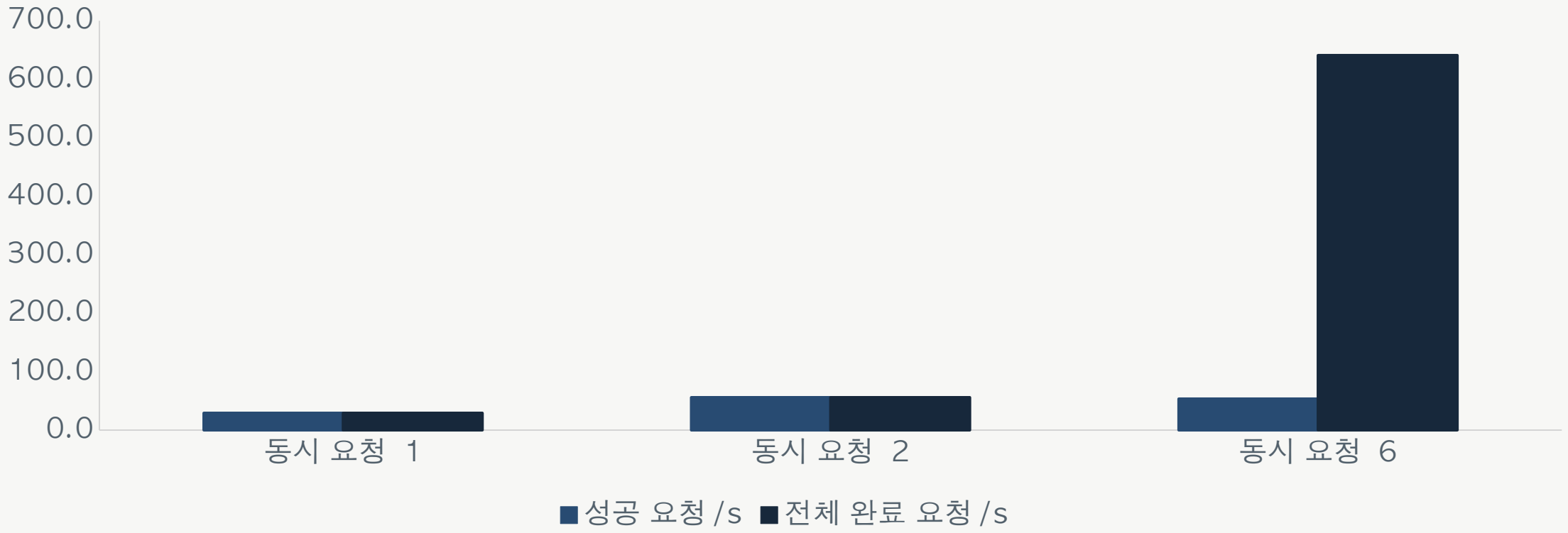
p95 · 관측값의 95백분위 지점

모든 요청이 95ms라는 의미도, 95%의 성공률도 아님

소수 표본의 p99로 장시간 서비스 목표를 보장하지 않음

실습은 NumPy percentile · linear 보간 · 실패율은 별도 분모

성공 처리량과 완료 처리량



성공 요청/s · 전체 완료 요청/s를 구분 · 빠른 거절은 성공 처리량에 포함하지 않음

성능 지표와 서비스 목표

SLI · 서비스 지표

Service Level Indicator

성공률·지연·완료 여부

SLO · 서비스 목표

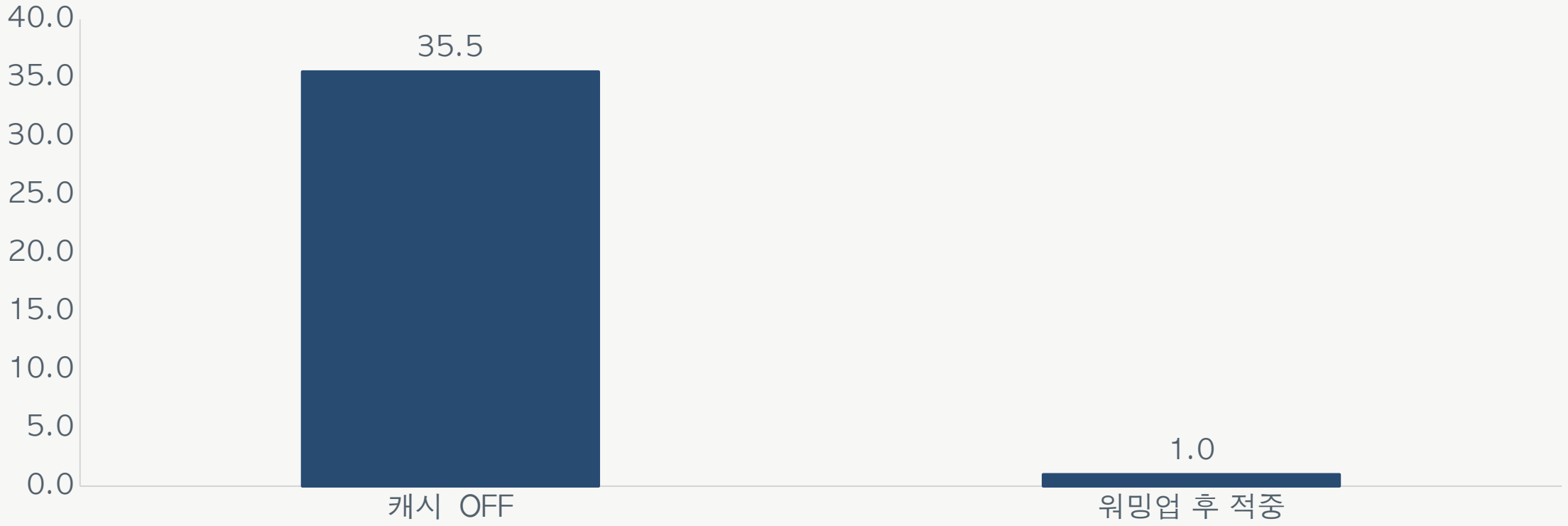
Service Level Objective

지표의 목표·기간·대상 정의

SLA · Service Level Agreement · 계약·책임의 합의 · 이번 실험은 SLA가 아님

요청·응답 계약 · 비동기 처리 · 실패 복구 · 비교 실험

캐시 적중과 비적중의 비교



성공 p50 · ms · 동시성 2 · 워밍업 제외 · 캐시 적중을 모델 속도로 해석하지 않음

결과와 함께 남길 실행 조건

구분	기록
환경	Python·패키지·기기·프로세스 수
입력	표본 수·길이·출력 제한·질의 구성
캐시	콜드·워밍업·적중·만료
부하	동시성·도착 방식·실행 시간
결과	성공 지연·오류율·성공 처리량·한계

결과 JSON은 로컬 output/validation/week09-lab/ · 공개 제출 원문과 분리

프로젝트 적용과 선택 확장

요구사항·API·시험·실패 처리의 연결

실제 모델 연결은 공통 경로 이후의 선택

기획서 요구사항의 실행 연결

항목	시험	실패 처리
FR01 · 근거 안내	정상·근거 부족 사례	검토 안내
FR02 · 구조화 응답	필드·타입·경계 검사	입력 수정·재시도 제한
NFR01 · 서비스 품질	지연·오류·완료를 같이 확인	용량·시간·복구 조정

FR · Functional Requirement / NFR · Non-Functional Requirement · 목표값은 제안 단계

Ollama 선택 경로의 사전 점검

01	**설치 확인**	ollama list · 정확한 모델 태그
02	**서버 확인**	127.0.0.1:11434 · /api/tags
03	**생성 확인**	think:false · 출력 제한 · 타임아웃

모델·서버 버전에 따라 thinking 지원 다름 · 실제 모델 속도는 기기에서 확인

실제 모델 연결의 명시적 설정

```
# 기존 MOCK 서버 종료 후 별도 터미널
export WEEK09_PROVIDER=ollama
export WEEK09_MODEL="설치된_정확한_모델_태그"
python -m uvicorn week09.lab.app:app \
  --host 127.0.0.1 --port 8009
```

예시 태그를 그대로 쓰지 않음 · /ready 확인 · 실제 모델 스트리밍은 이번 앱에서 미지원

공통 실습과 선택 모델의 경계

검증된 공통 경로

MOCK·5개 Notebook·계약·복구

ASGI·실제 TCP·브라우저 실행

선택 Ollama 경로

어댑터 계약은 모의 전송으로 검사

실제 속도·품질은 기기에서 재확인

최소 기능 한 개의 설계 리뷰

01	**사용 가치**	누구의 어느 요청을 처리하는가
02	**검증 근거**	성공·오류·취소를 재현할 수 있는가
03	**운영 경계**	권한·자료·비용·복구 중 빠진 것은 무엇인가

다음 주: 배포 구조·실행 관찰·복구 · 새 과제 기한은 소속 공지 기준

구현 범위와 남은 운영 위험

현재 구현

입력·출력·오류·단일 프로세스 한도

로컬 실험·자원 해제·완료 계약

배포 전 추가 작업

인증·권한·요청 본문 크기·암호화

외부 저장·다중 worker·취소·관측

참고 자료와 재현 경로

자료	출처·실행
FastAPI	fastapi.tiangolo.com · 요청·응답·async
HTTP·SSE	RFC 9110 · WHATWG HTML
PagedAttention	arxiv.org/abs/2309.06180 · Figure 4·5
실습 코드	week09/lab/ · notebooks/01–05
출처·한계	week09/lecture/assets/SOURCES.md

2025 실습의 주제 계승 · 공통 환경·실패 복구·개인정보 경계를 재설계

요청·응답 계약 · 비동기 처리 · 실패 복구 · 비교 실험